System Health Monitor & Security Auditing Daemon

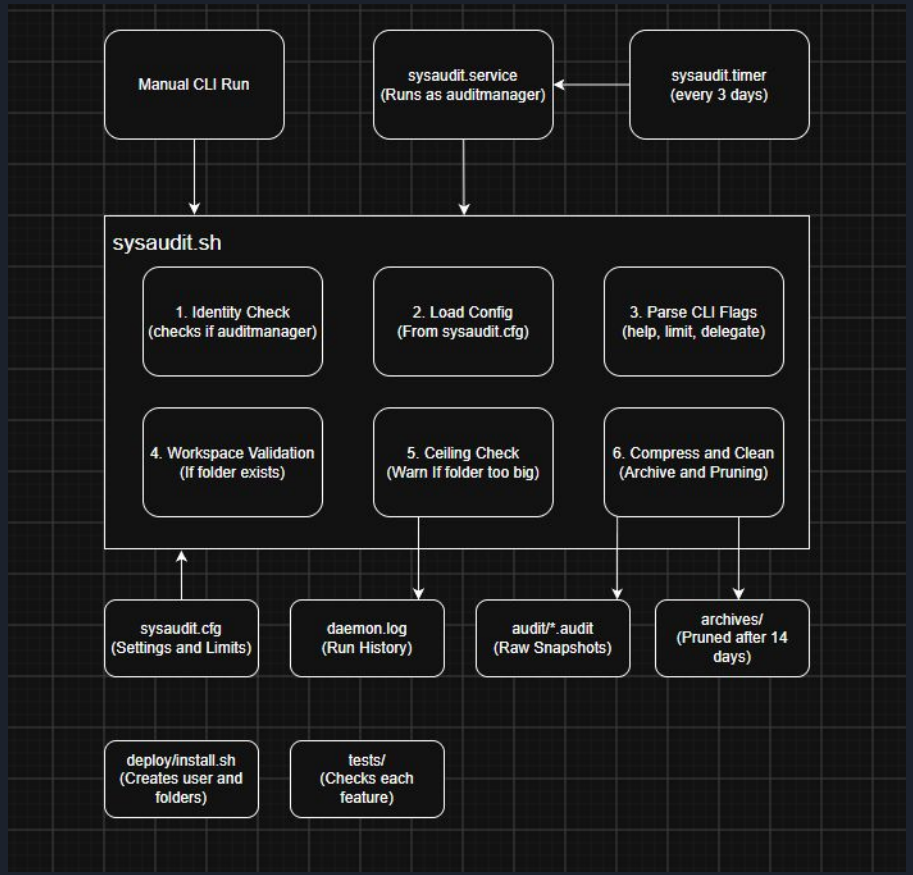
By: Braiden Dyck, Pavneet Gill, Brandon Millis, Vishesh Sharma, and Armaan Kandola

Table Of Contents

1. Overview
2. Basic Core Script
3. Daemon (systemd setup)
4. Security/User Access
5. Testing Suite
6. Challenges
7. Live Demo

Overview

Overview: Architecture Diagram



Overview: Lectures Referenced

We generally referenced every lecture, but some specifics:

User Management: Permissions

Scripting and the shell: Shell Script

The File System: POSIX

Basic Core Script

Basic Core Script

- Initializes the bash script
- Defines exit codes for different errors
- Handles errors and unexpected issues
- Displays logs error messages
- Ensures proper script termination

```bash
#!/bin/bash

# POSIX Exit Code(s)
E_SUCCESS=0
E_GENERIC=1
E_IDENTITY=2
E_CONFIG=3
E_WORKSPACE=4
E_CEILING=5

CONFIG_FILE="../config/sysaudit.cfg"

# Function for Exits & Logging
fail_with() {
  local code=$1
  local message=$2
  echo "[ERROR] $(date) - $message" >&2
  if [ -f "$LOG_FILE" ]; then
    echo "[ERROR] $(date) - Terminated with code $code: $message" >>"$LOG_FILE"
  fi
  exit "$code"
}
```

Basic Core Script: Cont.

- Verifies user identity before execution
- Restricts access to authorized users
- Prevents unauthorized script execution
- Checks for required configuration files
- Loads system settings from the configuration file

```
24  # User Execution Space Sandboxing
25  USER=$(id -un)
26  if [ "$USER" != "auditmanager" ]; then
27    fail_with "$E_IDENTITY" "Privilege violation: Process must execute as auditmanager."
28  fi
29  echo "Security Check passed! Welcome $USER."
30
31  # Configuration Loading
32  if [ ! -f "$CONFIG_FILE" ]; then
33    fail_with "$E_CONFIG" "Configuration file $CONFIG_FILE not found."
34  fi
35  source "$CONFIG_FILE"
```

Basic Core Script: Cont.

- Provides instructions for using the script
- Supports multiple command line options
- Allows user inputs and commands
- Identifies invalid commands and display errors

```
38  > display_help() { ...
80    }
81
82    #dump USAGE statement if run manually from the command line w/o args or help flag
83    if [[ "$#" -eq 0 || "$1" == "-h" || "$1" == "--help" ]]; then
84        display_help
85        exit 0
86    fi
87
88    #auditmanager permissions delegation section
89    DELEGATED_USER=""
90
91    while [[ "$#" -gt 0 ]]; do
92        case "$1" in
93            --limit)
94                STORAGE_LIMIT_MB="$2"
95                shift 2
96                ;;
97            --delegate)
98                #secure role delegation and autonomy allocation bit
99                DELEGATED_USER="$2"
100               shift 2
101               ;;
102           *)
103               echo "[ERROR] GENERIC_ERROR: Unknown parameter provided: $1" >&2
104               exit 1
105               ;;
106       esac
107   done
```

Daemon

Daemon (systemd setup)

- POSIX Exit Codes: Standardized numeric status codes for precise error reporting
- Identity Sandboxing: Enforces execution strictly under the *auditmanager* user
- Configuration & Environment: Sources parameters and verifies paths
- Auto-log Rotation: Compress raw audit files into timestamped *.tar.gz* archives
- Storage Pruning: Automatically purge archived files older than the retention threshold

```bash
#!/bin/bash

# POSIX Exit Code(s)
E_SUCCESS=0
E_GENERIC=1
E_IDENTITY=2
E_CONFIG=3
E_WORKSPACE=4
E_CEILING=5

CONFIG_FILE="/usr/local/bin/sysaudit.cfg"

# Function for Exits & Logging
fail_with() {
    local code=$1
    local message=$2
    echo "[ERROR] $(date) - $message" >&2
    if [ -f "$LOG_FILE" ]; then
        echo "[ERROR] $(date) - Terminated with code $code: $message" >> "$LOG_FILE"
    fi
    exit "$code"
}

# User Execution Space Sandboxing
if [ "$USER" != "auditmanager" ]; then
    fail_with "$E_IDENTITY" "Privilege violation: Process must execute as auditmanager."
fi

# Configuration Loading
if [ ! -f "$CONFIG_FILE" ]; then
    fail_with "$E_CONFIG" "Configuration file $CONFIG_FILE not found."
fi
source "$CONFIG_FILE"

# Default log file fallback if not defined in config
LOG_FILE="${LOG_FILE:-/var/log/sysaudit_daemon.log}"

# Workspace Validation
if [ ! -d "$TARGET_FOLDER" ]; then
    fail_with "$E_WORKSPACE" "Target workspace $TARGET_FOLDER missing or not a directory."
fi

echo " $(date) - Daemon execution started." >> "$LOG_FILE"
```

```bash
# Resource Ceiling Warning Check
CURRENT_SIZE_MB=$(du -sm "$TARGET_FOLDER" 2>/dev/null | cut -f1)
if [ -n "$CEILING_LIMIT_MB" ] && [ "$CURRENT_SIZE_MB" -gt "$CEILING_LIMIT_MB" ]; then
    echo "[WARNING] $(date) - Disk ceiling reached: ${CURRENT_SIZE_MB}MB exceeds limit of ${CEILING_LIMIT_MB}MB" >> "$LOG_FILE"
fi

# Compress and Cleanup
ARCHIVE_DIR="${TARGET_FOLDER}archives/"
mkdir -p "$ARCHIVE_DIR"

RAW_COUNT=$(find "$TARGET_FOLDER" -maxdepth 1 -name "*.audit" | wc -l)

if [ "$RAW_COUNT" -gt 0 ]; then
    TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
    ARCHIVE_NAME="${ARCHIVE_DIR}audit_archive_${TIMESTAMP}.tar.gz"

    echo " $(date) - Found $RAW_COUNT raw audit files. Packaging into archive." >> "$LOG_FILE"

    # Compress raw files into tar.gz package
    find "$TARGET_FOLDER" -maxdepth 1 -name "*.audit" -print0 | tar --null -czf "$ARCHIVE_NAME" --files-from -

    if [ $? -eq 0 ]; then
        echo " $(date) - Compression successful. Deleting individual raw source files." >> "$LOG_FILE"
        find "$TARGET_FOLDER" -maxdepth 1 -name "*.audit" -delete
    else
        echo "[ERROR] $(date) - Compression failed. Raw files preserved." >> "$LOG_FILE"
    fi
fi

# Automated Storage Pruning
RETENTION_DAYS=$(( ${RETENTION_WEEKS:-2} * 7 ))
echo " $(date) - Pruning archives older than $RETENTION_DAYS days..." >> "$LOG_FILE"
find "$ARCHIVE_DIR" -name "*.tar.gz" -mtime +"$RETENTION_DAYS" -delete

echo " $(date) - Daemon loop completed successfully." >> "$LOG_FILE"
exit "$E_SUCCESS"
```

Daemon (systemd setup): Cont.

- Service (sysaudit.service): Configure execution paths, user → auditmanager for least privilege, and ensure restart on failure
- Timer Schedule (sysaudit.timer): Triggers 15 minutes after system boot (OnBootSec=15min) and recurs every 3 days (OnUnitActiveSec=3d)
- Catch up Process (Persistent=true): Ensure any missed executions run immediately if system was powered off during scheduled time
- System Integration: Dependencies managed via systemd targets (multi-user .target, timers.target)

```
[Unit]
Description=Timer for System Health Monitor and Security Daemon
Requires=sysaudit.service

[Timer]
OnBootSec=15min
OnUnitActiveSec=3d
Persistent=true

[Install]
WantedBy=timers.target
```

```
[Unit]
Description=System Health Monitor & Security Daemon
After=network.target

[Service]
Type=simple
User=auditmanager
ExecStart=/usr/local/bin/sysaudit.sh
Restart=on-failure
RestartSec=10s
StartLimitIntervalSec=60s
StartLimitBurst=3

[Install]
WantedBy=multi-user.target
```

Daemon (systemd setup): Cont.

- Centralized Configuration (sysaudit.cfg) : Separates operational parameters from the script logic for easier maintenance
- Storage and Resource Limits: Sets workspace directory (/course_project/audit) and enforces a 100mb disk ceiling (MAX_DISK_MB)
- Automated Log Management: Direct daemon logs to /var/log/sysaudit_daemon.log
- Retention and Cleanup Policy: Automates log archiving every 3 days, with purges of archives older than 14 days.

```
AUDIT_DIR=/course_project/audit
MAX_DISK_MB=100
LOG_FILE=/var/log/sysaudit_daemon.log
SCAN_INTERVAL=60
ARCHIVE_RETENTION_DAYS=14
CLEANUP_INTERVAL_DAYS=3
```

```
BraidenDyck@Ubuntu:~$ sudo systemctl status sysaudit.timer
[sudo: authenticate] Password:
Warning: The unit file, source configuration file or drop-ins of sysaudit.timer changed on disk. Run 'systemctl daemon-reload' to reload units.
● sysaudit.timer - Timer for System Health Monitor and Security Daemon
     Loaded: loaded (/etc/systemd/system/sysaudit.timer; enabled; preset: enabled)
     Active: active (waiting) since Fri 2026-11-06 19:20:19 UTC; 3h 9min ago
 Invocation: 9a2a4148075145c186c77e13c71beb62
    Trigger: Mon 2026-11-09 19:35:16 UTC; 2 days left
   Triggers: ● sysaudit.service

Nov 06 19:20:19 Ubuntu systemd[1]: Started sysaudit.timer - Timer for System Health Monitor and Security Daemon.
```

Security/User Access

Security/User Access: Sandboxing

- Gets active username of person running script via (id -un) command
- Compares captured user with requirement
- Redirects error message to standard error stream if names don’t match
- Kills script completely with identity error to block further execution

```bash
#!/bin/bash

# POSIX Exit Code(s)
E_SUCCESS=0
E_GENERIC=1
E_IDENTITY=2
E_CONFIG=3
E_WORKSPACE=4
E_CEILING=5

CONFIG_FILE="/usr/local/bin/sysaudit.cfg"

# Function for Exits & Logging
fail_with() {
    local code=$1
    local message=$2
    echo "[ERROR] $(date) - $message" >&2
    if [ -f "$LOG_FILE" ]; then
        echo "[ERROR] $(date) - Terminated with code $code: $message" >> "$LOG_FILE"
    fi
    exit "$code"
}

# User Execution Space Sandboxing
USER=$(id -un)
if [ "$USER" != "auditmanager" ]; then
    fail_with "$E_IDENTITY" "Privilege violation: Process must execute as auditmanager."
fi
echo "Security Check passed! Welcome $USER."
```

Security/User Access: Usage Statement

- Checks if script was started with no arguments or the use of help flags

- Dumps detailed usage statement directly to terminal

```
#dump USAGE statement if run manually from the command line w/o args or help flag
if [[ "$#" -eq 0 || "$1" == "-h" || "$1" == "--help" ]]; then
 display_help
 exit 0
fi
```

Security/User Access: Usage Statement Cont.

- Shows all options for parameters, sample logs, and configuration schema

- Maps error code dictionary to human-readable exit descriptions

```
#detailed help statement and dumpin stuff
display_help() {
 cat <<'EOF'
==========================================================================
System Health Monitor & Security Auditing Daemon
==========================================================================



INSTRUCTIONS & OPERATIONAL SYNTAX:
 sysaudit [OPTIONS]
PARAMETER OPTIONS & EXECUTION EXAMPLES:
 sysaudit
 sysaudit -h
 sysaudit --help                 # Display this exhaustive USAGE statement
 sysaudit -r
 sysaudit --run
 sysaudit --scan                 # Run the manual scan and remove loop
 sysaudit -c
 sysaudit --cleanup              # Run only the archive pruning and cleanup
 sysaudit -d
 sysaudit --delegate tom_jones   # Delegate execution rights/file access to user 'tom_jones'
 sysaudit -w 50
 sysaudit --warn 50              # Override baseline storage limit to 50MB for this loop
 sysaudit -m 100
 sysaudit --max 100              # Override max storage limit to 100MB for this loop.



SAMPLE RESULTS & LOGS:
 [INFO] 2026-10-22 14:00:01 - Scan loop initialized successfully.
 [WARN] 2026-10-22 14:05:00 - Storage envelope at 95MB (Limit: 100MB).
 [ERROR] 2026-10-22 14:10:00 - IDENTITY_VIOLATION triggered.



CONFIGURATION SCHEMA (sysaudit.cfg):
 TARGET_DIR              : Target folder path for workspace (e.g., /course_project/audit/)
 CONFIG_DIR              : Target cfg path for workspace (default: ../config/sysaudit.cfg)
 WARN_DISK_MB            : Log over-warning limit in MB (default: 50MB)
 MAX_DISK_MB             : Over-allocation baseline limit in MB (default: 100MB)
 LOG_FILE                : Path to the dedicated internal auditing trace log (/var/log/sysaudit_daemon.log)
 SCAN_INTERVAL           : Interval for daemon scans (default: 60s)
 ARCHIVE_RETENTION_DAYS  : Number of days to retain archives (default: 14 days)
 CLEANUP_INTERVAL_DAYS   : Number of days between cleanup (default: 3 days)



ERROR CODE DICTIONARY:
 0 | SUCCESS               | Operational task ran to completion without tracking loops faults.
 1 | GENERIC_ERROR         | An unmapped error condition triggered a process escape.
 2 | IDENTITY_VIOLATION    | Process call called by an unverified account other than auditmanager.
 3 | CONFIGURATION_MISSING | The explicit target configuration layout file sysaudit.cfg was not found.
 4 | INVALID_WORKSPACE     | Target mapping path missing or is not a directory.
 5 | DISK_CEILING_REACHED  | Workspace capacity validation failed critical storage envelope margins.
EOF
}
```

Security/User Access: Argument Parser & User Delegation

- Set up defaults at beginning to hold customized settings
- Uses repeating loop to read through command line options input by user
- Identifies recognized flags and adjusts values accordingly
- Detects typos or unsupported commands, displays manual, and halts script with correct error

```
#auditmanager permissions delegation section
CLI_WARN_DISK_MB=""
CLI_MAX_DISK_MB=""
CLI_AUDIT_DIR=""
CLI_CONFIG_FILE=""
DELEGATED_USER=""
CLEANUP_FLAG=0
RUN_SCAN=0


while [[ "$#" -gt 0 ]]; do
 case "$1" in
 -h | --help)
   display_help
   exit 0
   ;;
 --config)
   CLI_CONFIG_FILE="$2"
   shift 2
   ;;
 --workspace)
   CLI_AUDIT_DIR="$2"
   shift 2
   ;;
 -w | --warn)
   CLI_WARN_DISK_MB="$2"
   shift 2
   ;;
 -m | --max | --limit)
   CLI_MAX_DISK_MB="$2"
   shift 2
   ;;
 -d | --delegate)
   DELEGATED_USER="$2"
   shift 2
   ;;
 -r | --run | --scan)
   RUN_SCAN=1
   shift 1
   ;;
 -c | --cleanup)
   CLEANUP_FLAG=1
   shift 1
   ;;
 *)
   display_help
   fail_with "$E_GENERIC" "GENERIC_ERROR: Unknown parameter in args: ${1}"
   ;;
 esac
done
```

Security/User Access: Argument Parser & User Delegation

- Runs background check for saved text to see if saved text matches existing user
- Changes group ownership of target directory to match requested account
- Updates folder permissions so new group has read, write and execute access

```
# Apply the delegation logic if the argument was passed
if [[ -n "$DELEGATED_USER" ]]; then

    # Verify the user exists on the system
    if id "$DELEGATED_USER" &>/dev/null; then
        echo "[INFO] Valid user found. Delegating file domain group access to: $DELEGATED_USER"

        #Change the group ownership of the workspace directory recursively
        chgrp -R "$DELEGATED_USER" "$TARGET_DIR" 2>/dev/null

        #Grant read, write, and execute permissions to the new group
        chmod -R g+rwx "$TARGET_DIR" 2>/dev/null


        echo "[INFO] Delegation successful. Workspace $TARGET_DIR is now accessible by $DELEGATED_USER."

    else
        # Trap the issue and document properly
        echo "[ERROR] GENERIC_ERROR: Delegated user '$DELEGATED_USER' does not exist on this system." >&2

        exit 1
    fi
fi
```

Testing Suite

Testing Suite

- Divided testing to match the responsibilities of each group member
    - ENGINE - Testing suite to test functionality of the script
    - ACCESS - Testing suite to test validity of user roles, identities, and permissions
    - DAEMON - Testing suite to test the validity of the daemon files and confirm functionality

```
brandon:suites/ (feature/testing) $ bash tests_main.sh engine
test_archive_and_pruning.bats
 ✓ scanning generates raw *.audit files inside audit workspace
 ✓ Maintenance cleanup of raw *.audit files into archives, deletion of raw audits.
 ✓ Compression failure preserves raw *.audit files and logs [ERROR]
 ✓ Storage pruning deletes compressed archives older than 14 days old
 ✓ Pruning retains archives 14 days old or newer
test_config.bats
 ✓ Custom WARN_DISK_MB in cfg dynamically alters warning threshold
 ✓ Missing sysaudit.cfg writes diagnostic to STDERR and exits 3
 ✓ Malformed sysaudit.cfg (invalid syntax) writes to STDERR, exits 3
 ✓ Malformed sysaudit.cfg (invalid assignment) writes to STDERR, exit 3
 ✓ CLI flag overrides config path && audit path
 ✓ Malfformed sysaudit.cfg (spaces around =)
 ✓ CLI flag overrides WARN_DISK_MB
test_logging.bats
 ✓ Transcript: creates self-monitoring log file at LOG_FILE path
 ✓ Transcript: scan loop init is logged with [INFO]
 ✓ Handled exceptions are recorded with [ERROR]
 ✓ Summary Log: total count of compressed archives is being recroded
 ✓ Summary log: records exact filename and byte size of largest archive
 ✓ data storage file dump is logged with [INFO]
 ✓ script sleep cycle config update is logged with [INFO]
 ✓ records modification timestamp of the largest archive
test_storage.bats
 ✓ Missing AUDIT_DIR workspace writes to log and exits 4 (INVALID_WORKSPACE)
 ✓ Check that AUDIT_DIR not pointing to a directory exits 4
 ✓ Normal workspace under WARN_DISK_MB baseline cfg runs cleanly
 ✓ Exceeding WARN_DISK_MB logs warning and exits 0
 ✓ Exceeding the MAX_DISK_MB ceiling writes to log and exits 5.
test_usage_and_cli.bats
 ✓ Running without arguments outputs USAGE statement and exits 0
 ✓ Running with -h triggers USAGE and exits 0
 ✓ Running with --help triggers usage and exits 0
 ✓ Malformed or unknown CLI flags write to STDERR and exit 1

29 tests, 0 failures

brandon:suites/ (feature/testing) $
```

```
@test "Check that AUDIT_DIR not pointing to a directory exits 4" {
  rm -rf "${MOCK_AUDIT_DIR}"
  touch "${MOCK_AUDIT_DIR%/}"

  run --separate-stderr "${MOCK_ENGINE}" --scan

  exit_invalid_workspace "$status"
  stderr_used
}

@test "Normal workspace under WARN_DISK_MB baseline cfg runs cleanly" {
  seed_audit_file "test_audit" "1 Days Ago" 512

  run "${MOCK_ENGINE}" --scan

  exit_success "$status"
  log_exists
  ! warning_in_log
}

@test "Exceeding WARN_DISK_MB logs warning and exits 0" {
  seed_large_files "$(( MOCK_WARN_DISK + 1 ))M"

  run "${MOCK_ENGINE}" --scan

  exit_success "$status"
  log_exists
  warning_in_log
}

@test "Exceeding the MAX_DISK_MB ceiling writes to log and exits 5." {
  seed_large_files "$(( MOCK_MAX_DISK + 5))M"

  run ${MOCK_ENGINE} --scan

  exit_disk_ceiling "$status"
  log_exists
  error_in_log
}
```

Testing Suite: ENGINE

```
test_identity_sandboxing.bats
 ✓ Execution successful for the auditmanager user
 ✓ Execution by unprivileged user rejects immediately and writes to STDERR
 ✓ Execution via root rejects access and exits 2
 ✓ Execution via sudo rejects access and exits
test_role_delegation.bats
 ✓ Auditmanager can delegate execution and file access to a regular user and exit 0
 ✓ Delegated secondary user can execute scan and access workspace without an access violation
 ✓ Unprivileged wannabe auditmanager is blocked, logged, and written to error
 ✓ Non-delegated secondary user is still blocked after another un-related user is delegated

8 tests, 0 failures

brandon:suites/ (feature/testing) $
```

```
@test "Non-delegated secondary user is still blocked after another un-related user is delegated" {
  mock_user "auditmanager"
  run "${MOCK_ENGINE}" --delegate "auditor_backup"
  exit_success "$status"

  mock_user "unlisted_user"
  run --separate-stderr "${MOCK_ENGINE}" --scan

  exit_identity_error "$status"
  stderr_used
}
```

Testing Suite: ACCESS

```
test_systemd_schedule.bats
 ✓ systemd/sysaudit.timer exists and contains [Timer] and [Install] Section
 ✓ sysaudit.timer schedules execution at midnight every 3 days
 ✓ sysaudit.timer is enabled for automatic timer target setup
 ✓ sysaudit.timer and .service target the audit workspace
test_systemd_service.bats
 ✓ systemd/sysaudit.service unit file exists and is not empty.
 ✓ sysaudit.service contains required [Unit] [Service] and [Install] sections
 ✓ sysaudit.service enforces execution as auditmanager
 ✓ sysaudit.service points ExecStart to sysaudit.sh daemon
 ✓ sysaudit.service launches in background multi-user mode

9 tests, 0 failures

brandon:suites/ (feature/testing) $
```

```
@test "sysaudit.service contains required [Unit] [Service] and [Install] sections" {
  grep -Eq "^\[Unit\]" "${SERVICE_FILE}"
  grep -Eq "^\[Service\]" "${SERVICE_FILE}"
  grep -Eq "^\[Install\]" "${SERVICE_FILE}"
}

@test "sysaudit.service enforces execution as auditmanager" {
  grep -Eq "^[[:space:]]*User=auditmanager([[:space:]]|$)" "${SERVICE_FILE}"
}
```

Testing Suite: DAEMON

Challenges

Challenges

- Failed status with sysaudit.timer - Daemon failed with error “start-limit-hit”

```
BraidenDyck@Ubuntu:/$ systemctl list-timers sysaudit.timer
NEXT LEFT LAST                         PASSED UNIT           ACTIVATES
-       - Mon 2026-11-02 21:58:47 UTC      - sysaudit.timer sysaudit.service

1 timers listed.
Pass --all to see loaded but inactive timers, too.
BraidenDyck@Ubuntu:/$ sudo systemctl status sysaudit.timer
× sysaudit.timer - Timer for System Health Monitor and Security Daemon
     Loaded: loaded (/etc/systemd/system/sysaudit.timer; enabled; preset: enabled)
     Active: failed (Result: start-limit-hit) since Tue 2026-11-03 17:39:06 UTC; 2h 15min ago
   Duration: 250ms
 Invocation: 5b81ce0bfe0d4f4db076a2df370671c4
    Trigger: n/a
   Triggers: ● sysaudit.service

Nov 03 17:39:06 Ubuntu systemd[1]: sysaudit.timer: Deactivated successfully.
Nov 03 17:39:06 Ubuntu systemd[1]: Stopped sysaudit.timer - Timer for System Health Monitor and Security Daemon.
Nov 03 17:39:06 Ubuntu systemd[1]: Stopping sysaudit.timer - Timer for System Health Monitor and Security Daemon...
Nov 03 17:39:06 Ubuntu systemd[1]: Started sysaudit.timer - Timer for System Health Monitor and Security Daemon.
Nov 03 17:39:06 Ubuntu systemd[1]: sysaudit.timer: Deactivated successfully.
Nov 03 17:39:06 Ubuntu systemd[1]: Stopped sysaudit.timer - Timer for System Health Monitor and Security Daemon.
Nov 03 17:39:06 Ubuntu systemd[1]: Stopping sysaudit.timer - Timer for System Health Monitor and Security Daemon...
Nov 03 17:39:06 Ubuntu systemd[1]: sysaudit.timer: Start request repeated too quickly.
Nov 03 17:39:06 Ubuntu systemd[1]: sysaudit.timer: Failed with result 'start-limit-hit'.
Nov 03 17:39:06 Ubuntu systemd[1]: Failed to start sysaudit.timer - Timer for System Health Monitor and Security Daemon.
BraidenDyck@Ubuntu:/$ sudo systemctl reset-failed sysaudit.timer
BraidenDyck@Ubuntu:/$ sudo systemctl daemon-reload
BraidenDyck@Ubuntu:/$ sudo systemctl restart sysaudit.timer
BraidenDyck@Ubuntu:/$ sudo systemctl status sysaudit.timer
```

- Running directory issues with sysaudit.sh

```
BraidenDyck@Ubuntu:~$ sudo -u auditmanager /usr/local/bin/sysaudit.sh
find: Failed to restore initial working directory: /home/user: Permission denied
find: Failed to restore initial working directory: /home/user: Permission denied
BraidenDyck@Ubuntu:~$ cd /
BraidenDyck@Ubuntu:/$ sudo -u auditmanager /usr/local/bin/sysaudit.sh
BraidenDyck@Ubuntu:/$ sudo tail -n 10 /var/log/sysaudit_daemon.log
[INFO] Tue Nov  3 07:28:58 PM UTC 2026 - Daemon execution started.
[INFO] Tue Nov  3 07:28:58 PM UTC 2026 - Pruning archives older than 14 days...
[INFO] Tue Nov  3 07:28:58 PM UTC 2026 - Daemon loop completed successfully.
[INFO] Tue Nov  3 07:29:40 PM UTC 2026 - Daemon execution started.
[INFO] Tue Nov  3 07:29:40 PM UTC 2026 - Pruning archives older than 14 days...
[INFO] Tue Nov  3 07:29:40 PM UTC 2026 - Daemon loop completed successfully.
```

Challenges: Cont.

Testing in a comprehensive testing suite like BATS winds up adding up to many lines of code.

```
brandon:suites/ (feature/testing) $ find . -name '*.bats' | xargs wc -l
   90 ./engine/test_logging.bats
   81 ./engine/test_archive_and_pruning.bats
   93 ./engine/test_config.bats
   62 ./engine/test_storage.bats
   40 ./engine/test_usage_and_cli.bats
   48 ./access/test_identity_sandboxing.bats
   58 ./access/test_role_delegation.bats
   39 ./daemon/test_systemd_service.bats
   41 ./daemon/test_systemd_schedule.bats
  552 total
```

Ensuring that functions have equivalents within the test suite that prove functionality was a very good way to get familiar with one command at a time.

```
id() {
  case "$1" in
  -un)
    echo "${MOCKED_USER}"
    return 0
    ;;
  -u)
    if [[ "${MOCKED_USER}" == "root" ]]; then
      echo "0"
    else
      echo "1000"
    fi
    return 0
    ;;
  -nG)
    local check_user="${2:-$MOCKED_USER}"
    local groups="users"
    if [[ "${check_user}" == "auditmanager" ]] || grep -qx "${check_user}" "${MOCK_GROUP_FILE}" 2>/dev/null; then
      groups="${groups} delegatedmanagers"
    fi
    echo "${groups}"
    return 0
    ;;
  *)
    if [[ -n "$1" && "$1" != -* ]]; then
      echo "uid=1001($1) gid=1001($1) groups=1001($1)"
      return 0
    fi
    command id "$@"
    ;;
  esac
}
```

Live Demo

The End

